
What is a Namespace in Kubernetes?

A **namespace** in Kubernetes is a logical boundary used to isolate and organize Kubernetes resources. It enables **multi-tenancy**, **resource segmentation**, and **access control** within a shared Kubernetes cluster.

Key Characteristics of Namespaces

Feature	Description
Isolation	Objects in one namespace are logically separated from others.
Scoping	Names of resources like Pod , Service , etc., must be unique within a namespace, not cluster-wide.
RBAC Integration	Different RoleBindings/ClusterRoleBindings can be applied to specific namespaces.
Resource Quotas	Enforce CPU, memory, and object count limits per namespace.
Network Policies	Define intra-namespace or inter-namespace traffic rules.

In-built (System) Namespaces in Kubernetes

A fresh Kubernetes cluster (especially from cloud providers or kubeadm) comes with a few **default system namespaces**:

Namespace	Description
default	The default namespace for user-created resources if no other namespace is specified.
kube-system	Contains Kubernetes system components like kube-dns, kube-proxy, kube-controller-manager, etc.

<code>kube-public</code>	Readable by all users (including unauthenticated); usually used to share publicly accessible data.
<code>kube-node-lease</code>	Contains Lease objects used by kubelets for heartbeat signaling (improves cluster scalability).

Let's go over these in more detail:

1. `default`

- **Purpose:** The fallback namespace for any object that doesn't explicitly specify a namespace.
- **Use Case:** Good for quick testing or small single-tenant environments.
- **Visibility:** User-visible and editable.

Example:

`kubectl get pods`

is equivalent to:

`kubectl get pods -n default`

2. `kube-system`

- **Purpose:** Hosts critical **system components** deployed by Kubernetes itself.
- **Includes:**
 - `kube-dns` or `CoreDNS` (cluster DNS)
 - `kube-proxy` (network routing)
 - `etcd` (if self-hosted)

- `kube-apiserver`, `kube-scheduler`, etc. (in self-managed clusters)

Important:

- Avoid deploying user workloads here.
 - Modifying resources here can break cluster functionality.
-

3. 🌐 `kube-public`

- **Purpose:** Used for **publicly readable** data.
 - **Visibility:** Accessible by all users, including unauthenticated users.
 - **Use Case:**
 - Often used to expose the cluster's public information, such as a public root certificate.
 - Not commonly used in most deployments, but still exists by default.
-

4. 🚦 `kube-node-lease`

- **Purpose:** Supports **lightweight heartbeats** from each node via `Lease` objects.
- **Benefit:** Improves **scalability** of the control plane by reducing the load on the `Node` resource.
- Each node periodically updates its own `Lease` in this namespace.

How it works:

- Instead of updating the Node object every few seconds, the kubelet just updates a small Lease object in `kube-node-lease`.

Additional Namespaces (if other tools are installed)

If you've installed additional tools or platforms, they might create their own namespaces. These are **not part of Kubernetes core**, but are commonly seen:

Namespace	Purpose
<code>ingress-nginx</code>	For NGINX ingress controller
<code>cert-manager</code>	For cert-manager (TLS certificates via ACME/Let's Encrypt)
<code>monitoring,</code> <code>prometheus</code>	For Prometheus, Grafana, Alertmanager, etc.
<code>istio-system</code>	Istio service mesh control plane components
<code>velero</code>	Backup and restore resources
<code>argocd</code>	For Argo CD (GitOps continuous delivery)
<code>openebs, rook-ceph</code>	For CSI and storage solutions

You can list all namespaces in your cluster with:

```
kubectl get namespaces
```

When Should You Create Your Own Namespace?

Use custom namespaces to:

- Separate **environments** (e.g., `dev`, `test`, `prod`)

- Isolate **teams** (e.g., `frontend`, `backend`, `data-science`)
- Organize **applications** (e.g., `app1`, `app2`)
- Apply **RBAC**, **resource quotas**, or **network policies** per namespace

Summary Table

Namespace	Type	Editable by Users	Used For
<code>default</code>	Built-in	 Yes	User resources when no namespace is specified
<code>kube-system</code>	Built-in	 Not recommended	Core system components
<code>kube-public</code>	Built-in	 Limited	Cluster-wide readable config
<code>kube-node-lease</code>	Built-in	 No	Node heartbeats via Lease objects
